

Estudo Dirigido — Programação Orientada a Objetos em C++

Baseado nas apostilas e exemplos estudados sobre Programação Orientada a Objetos (POO) em C++.

1) Explique as diferenças entre Programação Estruturada e Programação Orientada a Objetos.

Preencha a tabela:

Característica	Programação Estruturada	Programação Orientada a Objetos
Organização do código		
Foco principal		
Reutilização de código		
Uso de funções		
Uso de objetos		
Facilidade de manutenção		

2) Definições

Defina os seguintes conceitos:

- a) Classe
- b) Objeto
- c) Atributo
- d) Método
- e) Encapsulamento
- f) Herança
- g) Abstração
- h) Polimorfismo

3) Analise o código abaixo:

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    void setNome(string n) {  
        nome = n;  
    }  
    void setIdade(int i) {  
        idade = i;  
    }  
    void mostrar() {  
        cout << nome << " - " << idade << endl;  
    }  
};
```

Responda:

- a) Qual é o nome da classe?
- b) Quais são os atributos?
- c) Quais são os métodos?
- d) O código utiliza encapsulamento?

4) Private, Protected e Public

Explique a diferença entre:

- private
- protected
- public

Depois complete a tabela:

Modificador	Pode ser acessado na própria classe	Pode ser acessado pelas classes filhas	Pode ser acessado fora da classe
private			
protected			
public			

5) Correção de Código

O código abaixo possui problemas de encapsulamento.

```
class Conta {
public:
    double saldo;
};
```

- Explique o problema.
- Reescreva a classe corretamente utilizando encapsulamento.
- Crie métodos depositar() e sacar().

6) Crie uma classe chamada Produto contendo:

- nome
- preço
- quantidade

A classe deve possuir:

- construtor
- método para adicionar estoque
- método para remover estoque
- método para mostrar informações

7) Conceitual

Responda:

- O que é um construtor?
- Quando ele é executado?
- O que é um destrutor?
- Quando o destrutor é chamado?
- Qual a vantagem de usar construtores?

8) Exercício – Análise de Execução

Analise o código:

```
#include <iostream>
using namespace std;
class Teste {
public:
    Teste() {
        cout << "Construtor chamado" << endl;
    }
    ~Teste() {
        cout << "Destrutor chamado" << endl;
    }
};
int main() {
    Teste t1;
    return 0;
}
```

Responda:

- O que será exibido na tela?
- Em qual momento o destrutor é executado?

9) Rastreamento de Objetos (Tabela)

Analise o código abaixo:

```
#include <iostream>
using namespace std;
class Conta {
private:
    double saldo;
```

```

public:
    Conta() {
        saldo = 0;
    }
    void depositar(double valor) {
        saldo += valor;
    }
    void sacar(double valor) {
        saldo -= valor;
    }
    double getSaldo() {
        return saldo;
    }
};

int main() {
    Conta c;
    c.depositar(100);
    c.depositar(50);
    c.sacar(30);
    c.depositar(20);
    return 0;
}

```

Complete a tabela:

Linha executada	Operação	Valor do saldo
Criação do objeto	saldo = ?	
c.depositar(100)		
c.depositar(50)		
c.sacar(30)		
c.depositar(20)		

10) Rastreamento com Dois Objetos

Analise:

```

Conta a;
Conta b;
a.depositar(200);
b.depositar(500);
a.sacar(50);
b.depositar(100);

```

Complete:

Etapas	Saldo de a	Saldo de b
Inicial		
Após depositar em a		
Após depositar em b		
Após saque em a		
Após depósito em b		

11) Conceitual - Herança

Explique:

- O que é herança?
- O que é classe base (pai)?
- O que é classe derivada (filha)?
- Qual a vantagem da herança?

12) Identificação de Herança

Observe o código:

```

class Animal {
public:
    void comer() {
        cout << "Comendo";
    }
};

```

```
    }  
};  
class Cachorro : public Animal {  
};
```

Responda:

- a) Qual é a classe pai?
- b) Qual é a classe filha?
- c) O objeto da classe Cachorro pode usar o método comer()?
- d) Qual tipo de herança está sendo utilizada?

13) Implementação

Crie:

Classe pai:

Funcionario

Atributos:

- nome
- salario

Métodos:

- mostrarDados()

Classe filha:

Gerente

Novo atributo:

- bonus

Método:

- calcularSalarioFinal()

14) Análise de Código

Baseando-se no exemplo de ContaBancaria e ContaPoupanca.

Explique:

- a) Por que o atributo saldo foi declarado como protected?
- b) Qual vantagem isso traz para as classes filhas?
- c) O método sacar() foi reaproveitado? Como?